Quantized Serving Kernels and Dispatch

Deep Dive into SGLang — Chapter 19

Yin Yang

Foundation Books Project

Where this chapter sits

This is the first chapter of Volume II, and it sets a rule the rest of the volume reuses: a smaller tensor is useful only while it still implements the same serving operator.

- Quantization stores small *codes* plus *metadata* instead of full floating-point weights.
- SGLang's contribution is not a new quantization algorithm — it is the serving boundary that makes AWQ, GPTQ, FP8, GGUF, and Marlin executable inside one runtime.
- The danger is shape-preserving: a packed tensor can have the expected shape and still compute the wrong function.

Goal: read a quantized layer and say what bytes are stored, what rule interprets them, and which kernel is allowed to consume the pair.

Roadmap

The contract

Representation and targets

Invariants

Loading and dispatch

Tradeoffs and evidence

The takeaway

The contract

Three objects the runtime keeps separate

A quantized layer is not one thing. Keep three objects distinct:

Stored payload $\widehat{W}$ the packed bytes: INT4 codes, FP8 blocks, GGUF-packed tensors, Marlin tiles.

Metadata m the rule that interprets the bytes: scale layout, zero point, packing factor, format tag.

Kernel path κ the specialized layout that is *eligible* to consume that $(\widehat{W}, m)$ pair.

The layer operator is the semantic object. The compressed tensor is just *one representation* it may consume:

$$y = x\, D_m(\widehat{W}).$$

$\widehat{W}$ is stored data; D_m is the dequant / unpack / scale-apply map; y is the same output shape the dense layer would produce.

The lifecycle every arrow must preserve

- Every arrow must preserve the *same interpretation* of the encoded state.
- The chapter walks the arrows: metadata vocabulary, targets, invariants, loading, dispatch, tradeoffs, measurement, handoff.

Load the compact state, choose a legal path, or fall back — before the model's meaning changes.

Representation and targets

One code, two meanings: why metadata travels with bytes

Eight logical weights, two INT4 groups. Group G_0: $s_0 = 0.05,\ z_0 = 8$. Group G_1: $s_1 = 0.20,\ z_1 = 7$. The *same* stored code $q = 10$:

$$\underbrace{0.05\,(10 - 8) = 0.10}_{G_0}, \qquad \underbrace{0.20\,(10 - 7) = 0.60}_{G_1}.$$

The affine reconstruction rule for an unsigned INT4 group:

$$\widetilde{w}_i = s_G\,(q_i - z_G), \qquad i \in G.$$

- s_G is the tick spacing; z_G is the code that lands on real zero.
- Two INT4 codes pack into one byte — but the byte alone never says which group's scale to use.

The group boundary is part of the operator, as executable metadata.

Four targets, four ownership boundaries

A runtime does not quantize "the model" — it quantizes state with different lifetimes.

Target	Validation point	Shape-preserving risk
Weights	loader + layer-method construction, before requests	right width, wrong decoded weights
Activations	batch-time scale build + kernel launch	stale scales applied to ordered rows
MoE state	fused / DeepEP runner construction and dispatch	expert consumes wrong layout
KV state	cache alloc, reuse, eviction, attention read	reused page decoded with wrong rule

The target decides which boundary must validate the metadata — weights at load, activations at the forward call, KV across decode steps.

Invariants

The most dangerous failures are shape-preserving

Eight weights, first group $s = 0.25,\ z = 8$, nibbles $[8, 10, 7, 12]$:

$$0.25 \cdot ([8, 10, 7, 12] - 8) = [0,\ 0.5,\ -0.25,\ 1.0].$$

Read the row as *one* 8-value group instead of two 4-value groups: rank, byte count, and output width all still pass — the second half is decoded with the wrong scale.

Invariant (Step-alignment)
Dequantization must interpret every packed value with the same scale, zero point, group size, and layout used when the checkpoint was produced.

Tensor shape is a guard, not a proof. Dynamic batching, CUDA-graph padding, and backend swaps may not change how packed values are decoded.

Loading and dispatch

Names that do different jobs: AWQ / GPTQ vs. Marlin

Checkpoint semantics

- **AWQ, GPTQ**: what the packed bytes *mean* — bit width, group size, scales, zero points, g_{idx}.
- **GGUF**: a file format carrying per-tensor `qweight\_type` tags.

Executable layout

- **Marlin**: a CUDA *kernel layout* that may consume a compatible AWQ/GPTQ checkpoint.
- It does *not* define checkpoint semantics.

The rule is not "AWQ vs. GPTQ in the abstract." It is whether the loaded representation, metadata, and kernel path preserve the intended layer operation efficiently.

From concept to code: dispatch reads the format tag

GGUF keeps the tag on the hot path — the tag *and* the row count M pick the branch.

from `python/sglang/srt/layers/quantization/gguf.py`

```
def fused_mul_mat_gguf(x, qweight, qweight_type):
    if qweight_type in UNQUANTIZED_TYPES:
        return x @ qweight.T
    if x.shape[0] <= mmvq_safe and qweight_type in MMVQ_QUANT_TYPES:
        y = ggml_mul_mat_vec_a8(qweight, x, qweight_type, qweight.shape[0])
    elif qweight_type in MMQ_QUANT_TYPES:
        y = ggml_mul_mat_a8(qweight, x, qweight_type, qweight.shape[0])
    elif qweight_type in DEQUANT_TYPES:
        weight = ggml_dequantize(qweight, qweight_type, *shape, x.dtype)
        y = x @ weight.T
    else:
        raise NotImplementedError(...)  # unsupported tag: reject, don't guess
```

Same output shape in every branch; the metadata contract is the difference. Unsupported tags fail loudly.

Tradeoffs and evidence

Marlin eligibility: the layout, not the name, is the speed

GPTQ layer $[K, N] = [4096, 4096]$, 4-bit, group 128. Packed vs. dense FP16:

$$4096 \cdot 4096 \cdot \tfrac{4}{8} = 8\,\text{MiB} \qquad \text{vs.} \qquad 4096 \cdot 4096 \cdot 2 = 32\,\text{MiB}.$$

Dense GPTQ-Marlin shape gate: N divisible by 64, K divisible by 128, K divisible by group size. Here all pass — keep the GEMM on the packed layout.

Now $N' = 4064$: not divisible by 64. The Marlin path is rejected; a fallback may expand to the 32 MiB dense matrix.

A slower fallback still counts in the benchmark — it is part of the cost of using that representation. And with tensor parallelism, the gate applies to the rank-local N_{part}, not the full N.

Measurement keeps the claim honest

A quantized-run record binds provenance, workload, correctness probe, kernel availability, and resource metrics to one path.

Metric	Dense	GPTQ-Int4	Claim boundary
Output tok/s	2,469	2,605	host effects mixed — not uniform
TTFT	331 ms	449 ms	first token slower
ITL	8.17 ms	5.47 ms	decode cadence improved
Loaded weights	0.13 GB	0.72 GB	**no** memory-saving claim

A speedup that reads the wrong scale group is not an optimization. Accuracy sits in the same budget as memory and latency.

The takeaway

What to carry forward

1. A quantized layer is **payload $\widehat{W}$ + metadata m + eligible kernel κ** — kept in one account.
2. The served operator must be the declared map $x \mapsto xD_{m,f}(\widehat{W})$; **shape is a guard, not a proof**.
3. **Targets decide ownership**: weights load-time, activations batch-time, MoE + expert layout, KV across decode steps.
4. AWQ/GPTQ are **checkpoint semantics**; Marlin/GGUF-kernels are **executable layouts** — not one interchangeable list.
5. Dispatch **chooses a legal kernel or falls back**; measurement proves the realized path against the right baseline.

Next: Chapter 20 turns adapters into runtime low-rank updates — another representation that must preserve the layer operator.